

ARCANE VAULT

Project Implementation Documentation

D&D Character Management Web Application

Tristan De La Borda

BCLC24

2026

Table of Contents

1. Project overview	5
1.1 Introduction	5
1.2 Project goals and objectives	5
1.3 Scope	5
1.4 Technical stack summary	5
2. Infrastructure provisioning	6
2.1 Terraform: Azure resource provisioning.....	6
2.2 Terraform: Proxmox VM provisioning	6
2.3 Ansible: on-premises configuration.....	6
2.3.1 WireGuard gateway (VM 1).....	6
2.3.2 PostgreSQL database (VM 2)	6
2.3.3 Management and monitoring (VM 3).....	6
2.4 Automated deployment pipeline.....	6
3. Containerisation and orchestration.....	7
3.1 Docker images.....	7
3.1.1 Backend Dockerfile	7
3.1.2 Frontend Dockerfile	7
3.2 Azure Container Registry.....	7
3.3 Kubernetes (AKS) deployment	7
3.3.1 Cluster configuration	7
3.3.2 Backend deployment.....	7
3.3.3 Frontend deployment	7
3.3.4 WireGuard deployment	7
3.3.5 Ingress and TLS.....	7
3.4 Horizontal Pod Autoscaler	8
4. Networking and security	9
4.1 Hybrid cloud connectivity (WireGuard VPN)	9
4.2 Network segregation (NetworkPolicies)	9
4.3 Firewall rules (UFW)	9
4.4 Container security and least privilege	10
4.5 TLS and encryption in transit	10
4.6 Data encryption at rest.....	10

4.7 Rate limiting	10
5. Identity and access management.....	12
5.1 Azure Entra ID integration	12
5.2 Frontend authentication (MSAL.js)	12
5.3 Backend token validation	12
5.4 Kubernetes cluster access control	12
5.5 Secrets management.....	13
6. Database	14
6.1 PostgreSQL setup and configuration	14
6.2 Schema design	14
6.3 Migration strategy	14
6.4 Data access layer	14
6.5 Redis caching layer	15
7. Project management	16
7.1 Methodology and tooling.....	16
7.2 Sprint overview	16
7.2.1 Sprint 0 (23 Feb - 6 Mar 2026).....	16
7.2.2 Sprint 1 (9 Mar - 11 Mar 2026).....	16
7.2.3 Sprint 2 (11 Mar - 17 Mar 2026).....	16
7.2.4 Sprint 3 (17 Mar - 23 Mar 2026).....	17
7.3 Time tracking	17
8. Monitoring and observability.....	18
8.1 Azure Arc monitoring	18
8.2 Azure Log Analytics	18
8.3 Application-level logging	18
8.4 Kubernetes health probes.....	18
9. Feature implementation status	20
9.1 Infrastructure features.....	20
9.2 Security features.....	20
9.3 Application features	21
9.4 Additions beyond original proposal	21
10. Challenges and lessons learned	22
10.1 Technical challenges encountered	22
10.2 Lessons learned	22

10.3 Future improvements	22
11. Conclusion	23
12. References and appendices	24
12.1 References	24
12.2 Appendix A: configuration files	24
12.3 Appendix B: API reference	24

1. Project overview

1.1 Introduction

Guidance: Describe the Arcane Vault project, its purpose, and the problem it solves for D&D campaign management.

[To be completed]

1.2 Project goals and objectives

Guidance: List the key goals: hybrid cloud architecture, automated infrastructure, character sheet management, beginner-friendly tools.

[To be completed]

1.3 Scope

Guidance: Define what is in scope and what was explicitly excluded (MFA, WAF). Reference the original proposal and scope changes.

[To be completed]

1.4 Technical stack summary

Guidance: Summarise the technologies used across frontend, backend, and infrastructure layers.

Layer	Technologies
Frontend	React, MSAL.js, Nginx
Backend	Node.js, Express, PostgreSQL (pg), Redis
Infrastructure	Terraform, Ansible, Docker, Kubernetes (AKS)
Cloud services	Azure AKS, ACR, Entra ID, Log Analytics, Azure Arc
Networking	WireGuard VPN, Nginx Ingress, UFW
On-premises	Proxmox VMs (PostgreSQL, WireGuard gateway, management)

2. Infrastructure provisioning

2.1 Terraform: Azure resource provisioning

Guidance: Detail the Terraform configuration: AKS cluster, ACR, resource group, Entra ID app registration, Log Analytics, and state backend bootstrap.

[To be completed]

2.2 Terraform: Proxmox VM provisioning

Guidance: Explain the 3-VM Proxmox setup (WireGuard gateway, PostgreSQL, management) provisioned via Terraform with the bpg/proxmox provider.

[To be completed]

2.3 Ansible: on-premises configuration

2.3.1 WireGuard gateway (VM 1)

Guidance: Document the WireGuard installation, key generation, IP forwarding, iptables NAT rules, and UFW firewall rules.

[To be completed]

2.3.2 PostgreSQL database (VM 2)

Guidance: Document PostgreSQL 16 installation, database/user creation, pg_hba.conf rules, performance tuning parameters, and firewall config.

[To be completed]

2.3.3 Management and monitoring (VM 3)

Guidance: Document Azure Arc agent installation, connection via service principal, and monitoring utilities deployed.

[To be completed]

2.4 Automated deployment pipeline

Guidance: Explain the end-to-end deploy.sh script: Terraform apply, Ansible playbook, Docker build/push, AKS deploy, DB migration.

[To be completed]

3. Containerisation and orchestration

3.1 Docker images

3.1.1 Backend Dockerfile

Guidance: Explain the Node.js Alpine image, production dependencies, health check, and non-root user configuration.

[To be completed]

3.1.2 Frontend Dockerfile

Guidance: Explain the multi-stage build: Node build stage, Nginx serving stage, build args for environment variables.

[To be completed]

3.2 Azure Container Registry

Guidance: Document ACR setup, AKS-to-ACR role assignment (AcrPull), and image push workflow.

[To be completed]

3.3 Kubernetes (AKS) deployment

3.3.1 Cluster configuration

Guidance: Detail AKS cluster settings: node pool auto-scaling, Azure CNI networking, RBAC integration with Entra ID.

[To be completed]

3.3.2 Backend deployment

Guidance: Document the backend deployment spec, resource limits, liveness/readiness probes, ConfigMap and Secret references.

[To be completed]

3.3.3 Frontend deployment

Guidance: Document the frontend deployment spec, Nginx serving, resource limits, and health probes.

[To be completed]

3.3.4 WireGuard deployment

Guidance: Document the WireGuard sidecar in AKS: init container for sysctl, iptables NAT rules, port forwarding to on-prem VMs, K8s services exposing DB and SSH.

[To be completed]

3.3.5 Ingress and TLS

Guidance: Document NGINX Ingress Controller setup, cert-manager, TLS termination, rate limiting annotations, and routing rules.

[To be completed]

3.4 Horizontal Pod Autoscaler

Guidance: Document HPA configuration: CPU (70%) and memory (80%) targets, min/max replicas, scaling behaviour.

[To be completed]

4. Networking and security

4.1 Hybrid cloud connectivity (WireGuard VPN)

The hybrid cloud architecture connects Azure AKS to on-premises Proxmox infrastructure through a WireGuard VPN tunnel. This provides encrypted, low-latency connectivity between the cloud-hosted application containers and the on-premises database and management servers.

On the AKS side, a dedicated WireGuard deployment runs as a pod within the `arcane-vault` namespace. An init container enables IP forwarding via `sysctl`, and the main container uses the `linuxserver/wireguard` image with `NET_ADMIN` and `SYS_MODULE` capabilities. The pod is assigned the tunnel address `10.100.0.1/24` and connects to the Proxmox WireGuard gateway at `10.100.0.2/24`. PostUp iptables rules handle DNAT for PostgreSQL traffic (port 5432) and SSH access to all three on-premises VMs (ports 10022, 10122, 10222), with `MASQUERADE` on the `wg0` interface for return routing.

On the Proxmox side, VM 1 acts as the WireGuard gateway. Ansible generates key pairs during provisioning, configures the `wg0` interface, enables IP forwarding, and sets up UFW rules to allow forwarding of PostgreSQL traffic from the `10.100.0.0/24` tunnel subnet to the database VM. `PersistentKeepalive` is set to 25 seconds to maintain the tunnel through NAT.

Three Kubernetes ClusterIP services expose the tunnel endpoints within the cluster: `wireguard-db` (port 5432 to the PostgreSQL VM), `wireguard-wg-vm`, `wireguard-db-vm`, and `wireguard-mgmt-vm` (SSH access to each on-premises VM). The backend pods connect to PostgreSQL via the `wireguard-db` service name, abstracting the VPN routing entirely.

4.2 Network segregation (NetworkPolicies)

Kubernetes NetworkPolicies enforce microsegmentation between application components within the `arcane-vault` namespace. Two policies are defined:

Backend policy: Restricts ingress to port 5000 from pods labelled component: frontend and from the ingress-nginx namespace only. Egress is limited to DNS (port 53 UDP/TCP), PostgreSQL (port 5432 via the WireGuard tunnel), and HTTPS (port 443 for Entra ID token validation). All other inbound and outbound traffic is denied by default.

Frontend policy: Restricts ingress to port 80 from the ingress-nginx namespace only. No egress restrictions are applied as the frontend serves static assets and makes no outbound connections from the container.

This ensures that even if a container is compromised, lateral movement within the cluster is constrained. The backend cannot reach arbitrary external endpoints, and the frontend cannot communicate directly with the database or other internal services.

4.3 Firewall rules (UFW)

All three Proxmox VMs run UFW with a default deny policy. Each VM has specific allow rules tailored to its function:

WireGuard gateway (VM 1): Allows SSH (port 22/tcp), WireGuard (port 51827/udp), and route-forwarded PostgreSQL traffic (port 5432/tcp) from the `10.100.0.0/24` tunnel subnet to the database VM.

PostgreSQL server (VM 2): Allows SSH only from the management VM and the Ansible controller (10.0.96.162). PostgreSQL (port 5432/tcp) is allowed from the WireGuard VM and the tunnel subnet only. This is the most locked-down VM with no public-facing ports.

Management server (VM 3): Allows SSH (port 22/tcp) for administrative access. Outbound HTTPS is required for Azure Arc agent communication.

4.4 Container security and least privilege

All application containers follow the principle of least privilege. The backend Dockerfile specifies USER node, ensuring the Express server runs as a non-root user. Both frontend and backend deployments have explicit resource requests and limits defined, preventing any single pod from consuming excessive cluster resources.

The WireGuard container is the only pod granted elevated capabilities (NET_ADMIN and SYS_MODULE), which are required for creating tunnel interfaces and loading kernel modules. Its init container runs as privileged solely to set the net.ipv4.ip_forward sysctl parameter, after which the main container runs with only the specified capabilities.

Resource limits are set as follows: backend pods are capped at 500m CPU and 512Mi memory, frontend pods at 200m CPU and 256Mi memory, and the WireGuard pod at 200m CPU and 128Mi memory. These limits prevent resource exhaustion and enable the HPA to make accurate scaling decisions.

4.5 TLS and encryption in transit

All external traffic to the application is encrypted using TLS. The NGINX Ingress Controller terminates TLS connections using certificates managed by cert-manager with a self-signed ClusterIssuer. The ingress annotation nginx.ingress.kubernetes.io/ssl-redirect is set to true, forcing all HTTP traffic to HTTPS.

The WireGuard VPN tunnel provides encryption for all traffic between AKS and the on-premises infrastructure, including database queries and SSH management sessions. PostgreSQL connections through the tunnel use scram-sha-256 authentication, adding an application-level authentication layer on top of the encrypted transport.

Internal cluster traffic between pods (frontend to backend, backend to WireGuard service) travels over the Azure CNI virtual network. While not encrypted at the pod-to-pod level, the NetworkPolicies ensure only authorised pods can initiate these connections.

4.6 Data encryption at rest

AKS node disks are encrypted using Azure-managed server-side encryption (SSE) with platform-managed keys by default. This provides transparent encryption for all data stored on the cluster nodes, including container images and ephemeral storage.

The on-premises PostgreSQL database relies on filesystem-level security (restricted access via UFW and WireGuard tunnel isolation) rather than application-level encryption at rest. Data at the storage layer is protected by the Proxmox host access controls and the VM isolation provided by the hypervisor.

4.7 Rate limiting

Rate limiting is applied at two layers to protect against abuse and denial-of-service attacks.

Ingress layer: The NGINX Ingress Controller enforces a limit of 20 requests per second with a burst multiplier of 5, configured via annotations (`nginx.ingress.kubernetes.io/limit-rps` and `limit-burst-multiplier`). The proxy body size is capped at 5MB.

Application layer: Express rate-limit middleware on all `/api/` routes allows a maximum of 100 requests per 15-minute window per IP address. Standard rate limit headers are returned to clients, and legacy headers are disabled.

5. Identity and access management

5.1 Azure Entra ID integration

The application uses Azure Entra ID (formerly Azure Active Directory) as its identity provider. The Entra ID application registration is provisioned automatically via Terraform using the `azuread_application` resource. The registration configures OAuth2 implicit grant flow with both access token and ID token issuance enabled, and requests Microsoft Graph API permissions for `User.Read`, `openid`, `email`, and `profile` scopes.

Terraform also creates a service principal (`azuread_service_principal`) and an application password (`azuread_application_password`) with an expiry date of December 2027. Redirect URIs are configured as a variable to support both local development (`http://localhost:3000`) and production (`https://arcanevault.example.com`) environments.

5.2 Frontend authentication (MSAL.js)

The React frontend integrates the Microsoft Authentication Library (MSAL) using the `@azure/msal-browser` and `@azure/msal-react` packages. The MSAL configuration is defined in `authConfig.js`, specifying the client ID, authority (tenant-specific `login.microsoftonline.com` endpoint), redirect URI, and session storage for the token cache.

Authentication follows the popup login flow via `MsalProvider` wrapping the entire application. The `AuthContext` component manages the authentication lifecycle: login triggers `instance.loginPopup()`, token acquisition first attempts `acquireTokenSilent()` for seamless renewal and falls back to `acquireTokenPopup()` if silent acquisition fails, and logout calls `instance.logoutPopup()`. Protected routes use the `useIsAuthenticated()` hook to gate access to character management pages.

Upon successful authentication, the `syncUser()` function sends the access token to the backend `/api/auth/login` endpoint, which creates or updates the user record in PostgreSQL based on the Entra ID object ID (OID) claim.

5.3 Backend token validation

The backend validates incoming JWT tokens using the `jsonwebtoken` and `jwks-rsa` libraries. The JWKS client is configured to fetch signing keys from the Entra ID discovery endpoint (`https://login.microsoftonline.com/{tenant}/discovery/v2.0/keys`) with caching enabled (24-hour TTL) and rate limiting to prevent excessive key fetches.

Token verification checks three parameters: the audience must match the application client ID, the issuer must match the tenant-specific Entra ID v2.0 endpoint, and the algorithm must be RS256. On successful verification, the middleware extracts the user OID, preferred username (email), and display name from the token claims and attaches them to the request object.

A development bypass exists for local testing: when `NODE_ENV` is set to `development` and the `x-dev-user` header is present, the middleware injects a demo user identity without requiring a valid Entra ID token.

5.4 Kubernetes cluster access control

The AKS cluster has Azure RBAC enabled via the `azure_active_directory_role_based_access_control` block in Terraform, with `azure_rbac_enabled` set to true and the tenant ID bound to the Azure subscription tenant. This controls which Entra ID users and groups can perform administrative operations against the Kubernetes API (deploying pods, managing namespaces, viewing resources) through `kubectl`.

This is separate from the application-level access control. The application does not use Kubernetes RBAC for user authorisation. Instead, character access is enforced at the application layer through ownership checks: database queries filter by `user_id` to ensure users can only modify their own characters, while public characters are readable by anyone.

5.5 Secrets management

Secrets are managed through two mechanisms depending on the environment layer:

Kubernetes Secrets: Database credentials (`DB_USER`, `DB_PASSWORD`), Entra ID tenant and client IDs are stored as Kubernetes Secrets in the `arcane-vault` namespace. These are auto-generated by Terraform using template files and referenced by the backend deployment via `secretRef`.

Ansible Vault: Sensitive variables for on-premises configuration (database passwords, Azure Arc service principal secrets, WireGuard keys) are encrypted using Ansible Vault in `group_vars/all/vault.yml`. These are decrypted at playbook runtime.

The original proposal specified Azure Key Vault with automatic key rotation. This was replaced with Kubernetes Secrets and Ansible Vault to reduce complexity and Azure resource costs. The trade-off is that secret rotation must be performed manually rather than automatically.

6. Database

6.1 PostgreSQL setup and configuration

PostgreSQL 16 runs on a dedicated Proxmox VM (VM 2), installed and configured via Ansible. The database listens on all interfaces (`listen_addresses = '*'`) to accept connections through the WireGuard tunnel, with access restricted via `pg_hba.conf` rules that only permit the `arcane_admin` user from the WireGuard VM IP and the 10.100.0.0/24 tunnel subnet using `scram-sha-256` authentication.

Performance tuning parameters are applied via Ansible: `max_connections` is set to 100, `shared_buffers` to 256MB, `effective_cache_size` to 512MB, `work_mem` to 4MB, `maintenance_work_mem` to 64MB, `wal_buffers` to 16MB, and `random_page_cost` to 1.1 (optimised for SSD storage). Slow queries exceeding 1 second are logged via `log_min_duration_statement`.

6.2 Schema design

The database consists of two tables with a one-to-many relationship:

users: Stores authenticated users with UUID primary keys (`gen_random_uuid()`), Entra ID object ID (`entra_oid`, unique), email, display name, optional avatar URL, and timestamps. The `entra_oid` column is indexed for fast lookup during token validation.

characters: Stores D&D character sheets linked to users via a foreign key (`user_id`) with ON DELETE CASCADE. Includes all standard character attributes: name, race, subrace, class, level (CHECK 1-20), six ability scores (CHECK 1-30 each), HP, AC, speed, proficiency bonus, and JSONB columns for skills, equipment, and features. The `is_public` boolean controls visibility in the Public Vault. Indexes exist on `user_id` and on `is_public` (partial index, WHERE `is_public = true`).

6.3 Migration strategy

Database migrations are handled by a standalone Node.js script (`src/config/migrate.js`) that runs `CREATE TABLE IF NOT EXISTS` and `CREATE INDEX IF NOT EXISTS` statements. This idempotent approach allows the migration to be run repeatedly without side effects. Migrations are executed after deployment by running the script inside a backend pod via `kubectl exec`.

There is no migration versioning system (such as Knex migrations or Flyway). The current schema is simple enough that the IF NOT EXISTS approach is sufficient, but this would need to be replaced with versioned migrations if the schema evolves significantly.

6.4 Data access layer

The backend connects to PostgreSQL using the `pg` library with a connection pool configured for up to 20 concurrent connections, a 30-second idle timeout, and a 5-second connection timeout. The pool handles automatic connection recycling and emits errors to `stderr`, triggering a process exit on unexpected pool errors to allow Kubernetes to restart the pod.

Data access follows the model pattern with two modules: `User.js` provides `findOrCreate` (upsert on Entra OID), `findById`, and `findByOid` methods. `Character.js` provides `findByUser`, `findById`, `findPublic`, `create`, `update`, `delete`, `toExport`, and `validateImport` methods. All database

operations use parameterised queries (\$1, \$2) to prevent SQL injection. JSON fields (skills, equipment, features) are automatically serialised/deserialised between JavaScript objects and PostgreSQL JSONB.

6.5 Redis caching layer

Guidance: Document the Redis deployment on AKS, what data is cached, TTL strategy, and how it reduces latency for user data fetching.

[To be completed]

7. Project management

7.1 Methodology and tooling

Project management was conducted using ClickUp, organised into sprints within the C2_DELTR workspace. Work was tracked using a task board with statuses: planning, backlog, in progress, on hold, and done. Tasks were assigned priorities (urgent, high, normal, none) and time was logged against individual tasks where applicable.

The project ran across four sprints from late February to late March 2026, following an iterative approach where each sprint built upon the deliverables of the previous one. Sprint planning was lightweight, with tasks created and reprioritised as the project evolved.

7.2 Sprint overview

Metric	Value
Total tasks tracked	84
Completed	44
In progress	7
Backlog	3
On hold	3
Planning	1
Completion rate	52%

7.2.1 Sprint 0 (23 Feb - 6 Mar 2026)

Sprint 0 focused on foundational setup and Azure security training. Key activities included establishing the Azure-to-Proxmox connectivity, setting up the initial PostgreSQL database on Proxmox, creating budget alerts, and completing AZ-500 security coursework to inform the project security architecture.

Task	Status	Priority
------	--------	----------

7.2.2 Sprint 1 (9 Mar - 11 Mar 2026)

Sprint 1 carried over items from Sprint 0 and added VPN setup and testing tasks. Several AZ-500 lab tasks were moved to backlog as the focus shifted toward infrastructure development. Accounting study tasks reflect the concurrent academic workload.

Task	Status	Priority
------	--------	----------

7.2.3 Sprint 2 (11 Mar - 17 Mar 2026)

Sprint 2 was the most productive sprint, delivering the core infrastructure: Terraform scripts, Ansible playbooks, service principals, Terraform state backend, controller and testing VMs, website creation, and initial Digital Ocean test deployments. Architecture diagrams and automation tasks spanned into Sprint 3.

Task	Status	Priority
------	--------	----------

7.2.4 Sprint 3 (17 Mar - 23 Mar 2026)

Sprint 3 focused on finalisation and deployment: testing Terraform and Ansible scripts against Azure, provisioning Proxmox VMs via Terraform, configuring VPN connectivity, building and pushing Docker images to ACR, setting up Ansible configurations on deployed infrastructure, and revising architecture diagrams. Kubernetes deployment remained in progress at the time of export.

Task	Status	Priority
------	--------	----------

7.3 Time tracking

Time was logged against selected tasks in ClickUp. The following tasks had recorded time:

Task	Sprint	Time logged
------	--------	-------------

8. Monitoring and observability

8.1 Azure Arc monitoring

The management VM (VM 3) is connected to Azure Arc via the azcmagent, providing hybrid visibility from the Azure Portal. The Arc agent is installed via Ansible using the official Microsoft install script (<https://aka.ms/azcmagent>) and connected non-interactively using a dedicated service principal with Azure Connected Machine Onboarding role assigned at the resource group scope.

The connection is configured with resource metadata tags (project=arcane-vault, role=management) for identification within the Azure Portal. Once connected, the VM appears as a hybrid resource in the Azure resource group alongside the AKS cluster, ACR, and Log Analytics workspace, providing a unified view of both cloud and on-premises infrastructure.

8.2 Azure Log Analytics

A Log Analytics workspace is provisioned via Terraform with the PerGB2018 SKU and a 30-day retention policy. The workspace is configured and available for ingesting logs from AKS and Arc-connected machines. The OMS agent integration with AKS was temporarily disabled in the Terraform configuration to reduce resource consumption during development, but the workspace remains ready for activation.

8.3 Application-level logging

The Express backend uses Morgan for HTTP request logging. In production mode (NODE_ENV=production), the combined format is used, which logs remote address, user, timestamp, method, URL, HTTP version, status code, response size, referrer, and user agent. In development mode, the dev format provides concise colour-coded output for faster debugging.

Helmet middleware adds security headers to all responses. A dedicated health check endpoint (/api/health) returns the application status and is used by both Kubernetes probes and external monitoring tools.

8.4 Kubernetes health probes

All deployments in the arcane-vault namespace have both liveness and readiness probes configured:

Component	Probe endpoint	Initial delay	Period
Backend (liveness)	/api/health:5000	15s	30s
Backend (readiness)	/api/health:5000	5s	10s
Frontend (liveness)	/:80	10s	30s
Frontend (readiness)	/:80	5s	10s

Liveness probes detect hung processes and trigger pod restarts. Readiness probes gate traffic routing, ensuring that pods only receive requests after they are fully initialised. Combined with the HPA, this provides automatic recovery from transient failures: a failed liveness check restarts the pod, and the HPA can scale additional replicas if existing pods are consistently unready.

9. Feature implementation status

The following table summarises the implementation status of all features from the original project proposal, along with new additions.

9.1 Infrastructure features

Feature	Status	Notes
Kubernetes orchestration (AKS)	Implemented	AKS with auto-scaling node pool
Docker containers	Implemented	Multi-stage builds, health checks
Terraform IaC	Implemented	Azure + Proxmox provisioning
Auto-scalable resources (HPA)	Implemented	CPU 70%, memory 80% targets
Hybrid cloud (Azure + on-prem)	Implemented	AKS + Proxmox via WireGuard
On-premises database	Implemented	PostgreSQL 16 on Proxmox VM
Ansible automation	Implemented	3-VM configuration playbook
State preservation / fault tolerance	Implemented	K8s probes, HPA, persistent volumes
Monitoring (Azure Arc)	Implemented	Replaced Prometheus with Azure Arc
Redis caching	Implemented	Added to AKS cluster

9.2 Security features

Feature	Status	Notes
Network segregation (NetworkPolicies)	Implemented	Backend + frontend policies
Container security / least privilege	Implemented	Non-root, resource limits
Entra ID for IAM	Implemented	Full MSAL + JWT validation
Azure RBAC on AKS	Implemented	Entra ID tenant binding
TLS / encryption in transit	Implemented	cert-manager + WireGuard
Firewall rules (UFW)	Implemented	All 3 VMs, deny-by-default
Rate limiting	Implemented	Nginx + Express layers
Secrets management	Partial	K8s Secrets + Ansible Vault (no Azure Key Vault)
Data encryption at rest	Partial	Azure-managed disk encryption only

Feature	Status	Notes
Multi-factor authentication	Not implemented	Excluded from scope
Web application firewall (WAF)	Not implemented	Excluded from scope

9.3 Application features

Feature	Status	Notes
Character sheets with editable fields	Implemented	Full CRUD with wizard
Validation checks	Implemented	Frontend + backend + DB constraints
Data persistence (PostgreSQL)	Implemented	Users + characters tables
User interaction logging	Partial	HTTP logging only (Morgan)
Session implementation	Partial	Public Vault exists, no real-time sessions
WebSocket real-time features	Not implemented	Not in current codebase

9.4 Additions beyond original proposal

Feature	Status	Notes
Character import/export (JSON)	Implemented	New feature
Public Vault (community gallery)	Implemented	New feature
WireGuard VPN tunnel	Implemented	New feature
Azure Arc integration	Implemented	New feature
Terraform state backend bootstrap	Implemented	New feature
Full automated deploy script	Implemented	New feature
D&D reference data	Implemented	New feature
Nginx ingress with rate limiting	Implemented	New feature

10. Challenges and lessons learned

10.1 Technical challenges encountered

Guidance: Document specific technical problems faced during development and how they were resolved.

[To be completed]

10.2 Lessons learned

Guidance: Reflect on what went well, what could be improved, and what you would do differently.

[To be completed]

10.3 Future improvements

Guidance: List potential enhancements: WebSocket sessions, Azure Key Vault integration, WAF, MFA, CI/CD pipeline, etc.

[To be completed]

11. Conclusion

Guidance: Summarise the project outcomes, key achievements, and how the implementation meets the original project goals.

[To be completed]

12. References and appendices

12.1 References

Guidance: List all documentation, tutorials, and resources used during the project.

[To be completed]

12.2 Appendix A: configuration files

Guidance: Include or reference key configuration files: Terraform variables, Ansible inventory, K8s manifests, Docker Compose.

[To be completed]

12.3 Appendix B: API reference

Guidance: Provide a complete API endpoint reference table with methods, paths, auth requirements, and descriptions.

[To be completed]